

Explicación detallada del mecanismo de decisión

Descripción general del sistema

El sistema implementa una API REST desarrollada con Flask que evalúa automáticamente entregas de estudiantes en formato PDF o texto plano.

El mecanismo de decisión se basa en:

- Representación semántica del texto mediante embeddings
- Cálculo de similitud coseno
- Aplicación de reglas con umbrales definidos

El modelo utilizado para generar embeddings es:

Hugging Face → modelo paraphrase-multilingual-MiniLM-L12-v2

Este modelo convierte cada texto en un vector de 384 dimensiones que captura su significado semántico.

Variables utilizadas en la decisión

♦ Variables principales

- `embedding_consigna` → vector de la consigna
- `embeddings_previos` → lista de vectores de respuestas anteriores
- `embedding_texto` → vector del texto entregado por el alumno

♦ Métricas calculadas

- `similitud_consigna` → Similitud coseno entre el texto del alumno y la consigna.
- `max_similitud_previa` → Mayor similitud coseno entre el texto del alumno y las respuestas previas almacenadas.

La similitud coseno toma valores entre:

- 1 → completamente opuesto
- 0 → sin relación
- 1 → idéntico

En este caso, como son embeddings semánticos, los valores suelen estar entre 0 y 1.

♦ Umbrales definidos

`UMBRAL_COPIA` = 0.95

`UMBRAL_TEMA` = 0.50

Interpretación:

- Si `similitud con respuesta previa` > 0.95 → Posible copia
- Si `similitud con consigna` < 0.50 → Fuera de tema
- En otro caso → Entrega válida

Algoritmo de decisión

El sistema aplica una lógica basada en reglas secuenciales.

Orden de evaluación:

1. Primero se detecta posible copia.
2. Luego se verifica si está fuera de tema.
3. Si no cumple ninguna condición anterior, se acepta.

Esto es importante porque la copia tiene prioridad sobre el fuera de tema.

Pseudocódigo final documentado

INICIO

Cargar modelo de embeddings

Definir CONSIGNA

Calcular embedding_consigna

Cargar respuestas_previas

Calcular embeddings_previos

Definir UMBRAL_COPIA = 0.95

Definir UMBRAL_TEMA = 0.50

RECIBIR texto del alumno

Si no hay texto:

devolver error

Calcular embedding_texto

Calcular similitud_consigna = cosine(embedding_texto, embedding_consigna)

Para cada embedding en embeddings_previos:

calcular similitud con embedding_texto

max_similitud_previa = máximo de similitudes_previas

Si max_similitud_previa > UMBRAL_COPIA:

decision = "Posible copia"

SINO Si similitud_consigna < UMBRAL_TEMA:

decision = "Fuera de tema"

SINO:

decision = "Entrega válida"

DEVOLVER similitud_consigna, max_similitud_previa, decision

FIN

Justificación del mecanismo

♦ **¿Por qué usar embeddings?**

Porque permiten comparar significado semántico y no solo palabras exactas.

Ejemplo:

- “Sumo la lista y divido entre su tamaño”
- “Uso sum() y len() para calcular el promedio”

Aunque no sean iguales textualmente, el modelo detecta que significan lo mismo.

♦ **¿Por qué usar similitud coseno?**

Porque mide el ángulo entre vectores, lo que permite evaluar qué tan similares son en el espacio semántico independientemente de la longitud del texto.

♦ **¿Por qué esos umbrales?**

Se obtuvieron mediante calibración experimental:

- 0.95 → detecta textos prácticamente idénticos
- 0.50 → separa textos relacionados del contenido irrelevante

Estos valores fueron ajustados probando con datos reales y evaluando precisión.

Análisis de casos límite

⚠ **Caso 1: Texto muy corto**

Ejemplo: “Sí” o “Promedio”

Problema:

- Puede generar embedding poco representativo.
- Puede dar similitudes artificialmente bajas.

Riesgo:

- Clasificación como “Fuera de tema” aunque el alumno haya intentado responder.

△ **Caso 2: Paráfrasis muy avanzada**

Si un alumno reescribe completamente la solución usando otra estructura:

- Podría bajar la similitud por debajo de 0.95
- No sería detectado como copia

Esto es una limitación de cualquier sistema basado en similitud semántica.

△ **Caso 3: Respuesta larga con relleno**

Si el alumno copia pero agrega mucho texto adicional:

- La similitud puede bajar ligeramente
- Podría no superar 0.95

△ **Caso 4: Tema relacionado pero incorrecto**

Un texto que hable de listas en Python pero no del promedio:

- Puede tener similitud > 0.50
- Ser clasificado como válido aunque no responda correctamente

El sistema evalúa relevancia semántica, no corrección lógica.

Ventajas del mecanismo

- Automatizable
- Rápido
- Escalable
- No depende de palabras exactas
- Detecta similitud semántica

Limitaciones

- No evalúa si el código funciona
- No detecta copia con modificaciones profundas
- Depende de umbrales calibrados
- No entiende contexto pedagógico

Conclusión técnica

El sistema implementa un clasificador basado en reglas determinísticas utilizando embeddings semánticos y similitud coseno.

La decisión final depende de:

- Comparación contra consigna (relevancia)
- Comparación contra entregas previas (originalidad)
- Umbrales calibrados experimentalmente

El modelo no aprende dinámicamente; funciona mediante lógica fija con reglas definidas.